

Jaden Vichob

Professor Eldon

ECE 111 - Advanced Digital Design

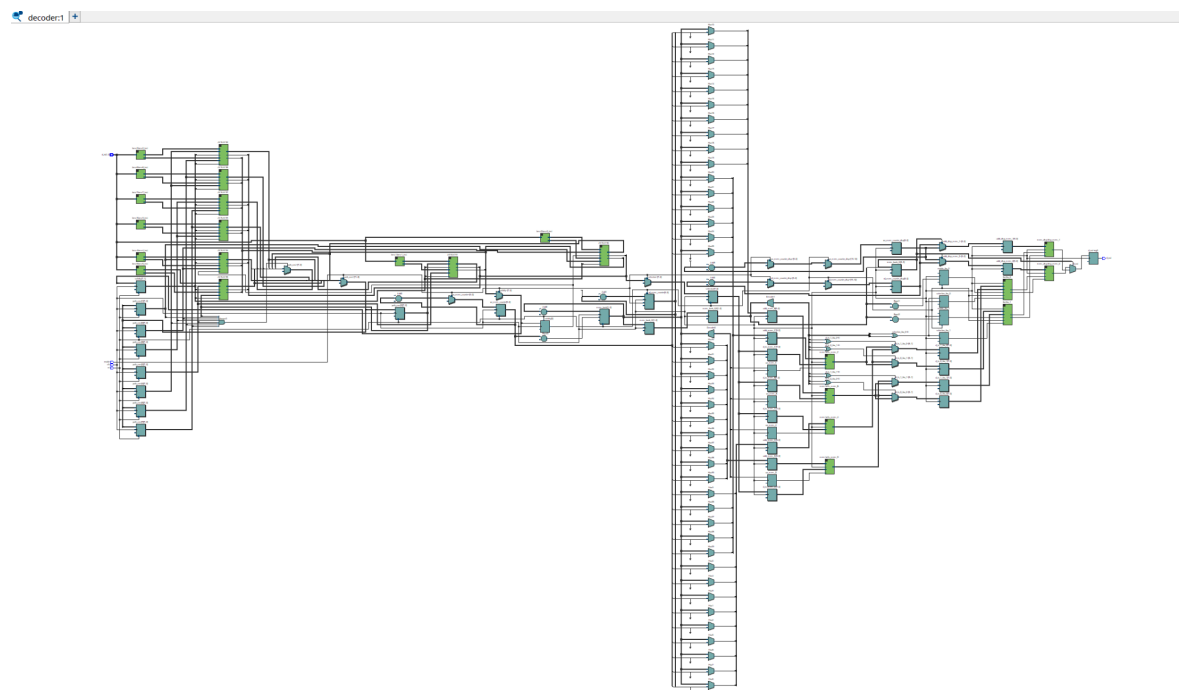
21 March 2025

Lab Report Term Project ECE-111

Part 1

1. Post-Synthesis Results

a. RTL Schematic

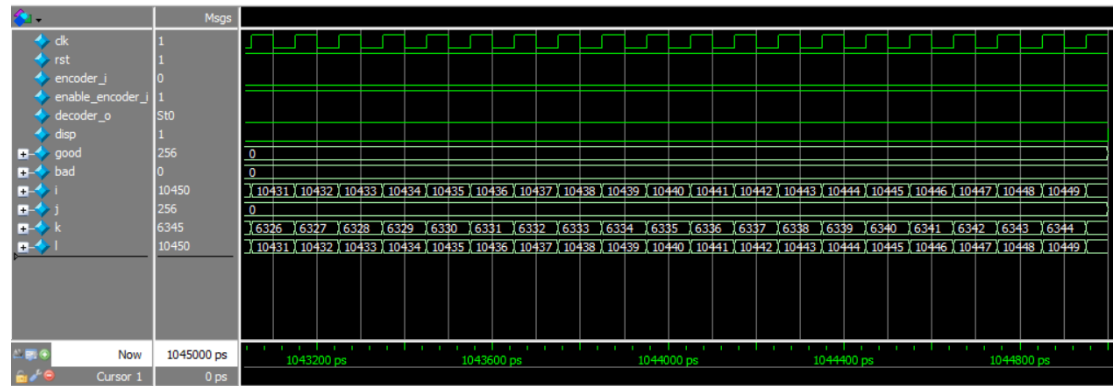


b. Utilization Report

	Resource	Usage
1	▼ Estimated ALUTs Used	358
1	-- Combinational ALUTs	358
2	-- Memory ALUTs	0
3	-- LUT_REGS	0
2	Dedicated logic registers	239
3		
4	▼ Estimated ALUTs Unavailable	54
1	-- Due to unpart...binational logic	54
2	-- Due to Memory ALUTs	0
5		
6	Total combinational functions	358
7	▼ Combinational ALU... number of inputs	
1	-- 7 input functions	8
2	-- 6 input functions	56
3	-- 5 input functions	16
4	-- 4 input functions	23
5	-- <=3 input functions	255
8		
9	▼ Combinational ALUTs by mode	
1	-- normal mode	182
2	-- extended LUT mode	8
3	-- arithmetic mode	168
4	-- shared arithmetic mode	0
10		
11	Estimated ALUT/register pairs used	486
12		
13	▼ Total registers	239
1	-- Dedicated logic registers	239
2	-- I/O registers	0
3	-- LUT_REGS	0
14		
15		
16	I/O pins	6
17	Total MLAB memory bits	0
18	Total block memory bits	34816
19		
20	DSP block 18-bit elements	0
21		
22	Maximum fan-out node	cl...ut
23	Maximum fan-out	273
24	Total fan-out	2547
25	Average fan-out	3.96

2. ModelSim Results

a. Waveform



b. Transcript

```
# yaa! in = 1, out = 1, w_ct = 216, err = 00
# yaa! in = 1, out = 1, w_ct = 217, err = 00
# yaa! in = 1, out = 1, w_ct = 218, err = 00
# yaa! in = 1, out = 1, w_ct = 219, err = 00
# yaa! in = 1, out = 1, w_ct = 220, err = 00
# yaa! in = 1, out = 1, w_ct = 221, err = 00
# yaa! in = 1, out = 1, w_ct = 222, err = 00
# yaa! in = 1, out = 1, w_ct = 223, err = 00
# yaa! in = 1, out = 1, w_ct = 224, err = 00
# yaa! in = 1, out = 1, w_ct = 225, err = 00
# yaa! in = 1, out = 1, w_ct = 226, err = 00
# yaa! in = 1, out = 1, w_ct = 227, err = 00
# yaa! in = 1, out = 1, w_ct = 228, err = 00
# yaa! in = 1, out = 1, w_ct = 229, err = 00
# yaa! in = 1, out = 1, w_ct = 230, err = 00
# yaa! in = 1, out = 1, w_ct = 231, err = 00
# yaa! in = 1, out = 1, w_ct = 232, err = 00
# yaa! in = 1, out = 1, w_ct = 233, err = 00
# yaa! in = 1, out = 1, w_ct = 234, err = 00
# yaa! in = 1, out = 1, w_ct = 235, err = 00
# yaa! in = 1, out = 1, w_ct = 236, err = 00
# yaa! in = 1, out = 1, w_ct = 237, err = 00
# yaa! in = 1, out = 1, w_ct = 238, err = 00
# yaa! in = 1, out = 1, w_ct = 239, err = 00
# yaa! in = 1, out = 1, w_ct = 240, err = 00
# yaa! in = 1, out = 1, w_ct = 241, err = 00
# yaa! in = 1, out = 1, w_ct = 242, err = 00
# yaa! in = 1, out = 1, w_ct = 243, err = 00
# yaa! in = 1, out = 1, w_ct = 244, err = 00
# yaa! in = 1, out = 1, w_ct = 245, err = 00
# yaa! in = 1, out = 1, w_ct = 246, err = 00
# yaa! in = 1, out = 1, w_ct = 247, err = 00
# yaa! in = 0, out = 0, w_ct = 248, err = 00
# yaa! in = 1, out = 1, w_ct = 249, err = 00
# yaa! in = 1, out = 1, w_ct = 250, err = 00
# yaa! in = 1, out = 1, w_ct = 251, err = 00
# yaa! in = 1, out = 1, w_ct = 252, err = 00
# yaa! in = 1, out = 1, w_ct = 253, err = 00
# yaa! in = 1, out = 1, w_ct = 254, err = 00
# yaa! in = 1, out = 1, w_ct = 255, err = 00
# corrupted_bits = 16, OUT: good = 256, bad = 0
```

3. Explanation of Encoder/Decoder

a. Encoder Explanation

The Viterbi Encoder utilizes a 3-bit shift register to manage 3 states in a finite state machine. Moving from the current state to the next state, the two MSBs are shifted to the right, and the new bit is calculated of the XOR of the input bit and the two LSBs of the current state ($d_in \wedge cstate[1] \wedge cstate[0]$). As the encoder is a rate $\frac{1}{2}$ convolutional encoder, the 1-bit input is turned into a 2-bit output sequence. The first output bit is the input bit, while the second output bit is the XOR of the input bit and the two MSB of the current state ($d_in \wedge cstate[2] \wedge cstate[1]$). The encoder also has enable and reset input signals, as well as a validity output signal to ensure that the output is 2-bits wide.

b. Decoder Explanation

The Viterbi Decoder is much more complicated than its encoder. There are three main components which make up the Viterbi Decoder, the add-compare-select (ACS), the branch metric computation block (BMC), and the trace back unit (TBU). When the decoder receives the output bits from the encoder, the eight BMCs check each state and compares the input bits to the expected bits for each state transition, where each state is evaluated in parallel. This effectively emulates a trellis, which models all eight possibilities. The ACS finds the lowest cost path out of the two possible states (0 or 1) based on the information from the BMCs, which then outputs this selection to a memory bank pipeline. This is repeated for the entirety of the input signal from the encoder, storing the selection signals to four memory banks: A, B, C, and D. The TBUs then trace through the lowest cost

path from the memory banks, determining the most likely transmitted sequence, and thus reconstructing the original input sequence to the encoder. The decoded bits are then stored in another memory bank pipeline, which selects the correctly decoded bit output.

Part 2

a) Even spacing of corrupted bits

1. Inverted bit[0] every 8 samples

Changed condition for errors to be injected if $\text{word_ct}[N-2:0] == 1$.

Yields total of 32 corrupted bits with all 256 inputs and outputs matching.

2. Inverted bit[1] every 8 samples

Assigned err_inj to 2'b10 from 2'b01 if $\text{word_ct}[N-2:0] == 1$.

Yields total of 32 corrupted bits with all 256 inputs and outputs matching.

3. Inverted both bit[0] and bit[1] every 16 samples

Changed condition for errors to be injected if $\text{word_ct}[N-1:0] == 1$.

Assigned err_inj to 2'b11.

Yields total of 16 corrupted bits with all 256 inputs and outputs matching.

4. Inverted bit[0] twice every 16 samples

Assigned error_inj to 2'b01 if $\text{word_ct}[N-1:1] == 1$.

Yields total of 32 corrupts bits with all 256 inputs and outputs matching.

5. Inverted bit[1] every 16 samples

Assigned error_inj to 2'b10 if $\text{word_ct}[N-1:1] == 1$.

Yields total of 32 corrupts with all 256 inputs and outputs matching.

6. Inverted bit[0] four times every 32 samples

Assigned error_inj to 2'b01 if $\text{word_ct}[N:2] == 1$.

Yields total of 32 corrupts with all 256 inputs and outputs matching.

7. Inverted bit[1] four times every 32 samples

Assigned error_inj to 2'b10 if word_ct[N:2] == 1.

Yields total of 32 corrupts with all 256 inputs and outputs matching.

8. Inverted bit[0] and bit[1] twice every 32 samples

Assigned error_inj to 2'b11 if word_ct[N:1] == 1.

Yields total of 16 corrupts with all 256 inputs and outputs matching.

b) Randomized spacing of corrupted bits

1. Inverted bit[0] every 8 samples

Changed condition for errors to be injected if err_trig[N-2:0] == 1.

Yields total of 26 corrupted bits with all 256 inputs and outputs matching.

2. Inverted bit[1] every 8 samples

Assigned err_inj to 2'b10 from 2'b01 if err_trig[N-2:0] == 1.

Yields total of 26 corrupted bits with all 256 inputs and outputs matching.

3. Inverted both bit[0] and bit[1] every 16 samples

Changed condition for errors to be injected if err_trig[N-1:0] == 1.

Assigned err_inj to 2'b11.

Yields total of 15 corrupted bits with all 256 inputs and outputs matching.

4. Inverted bit[0] twice every 16 samples

Assigned error_inj to 2'b01 if err_trig[N-1:1] == 1.

Yields total of 31 corrupts bits with all 256 inputs and outputs matching.

5. Inverted bit[1] every 16 samples

Assigned error_inj to 2'b10 if err_trig[N-1:1] == 1.

Yields total of 31 corrupts with all 256 inputs and outputs matching.

6. Inverted bit[0] four times every 32 samples

Assigned error_inj to 2'b01 if err_trig[N:2] == 1.

Yields total of 31 corrupts with all 256 inputs and outputs matching.

7. Inverted bit[1] four times every 32 samples

Assigned error_inj to 2'b10 if err_trig[N:2] == 1.

Yields total of 31 corrupts with all 256 inputs and outputs matching.

8. Inverted bit[0] and bit[1] twice every 32 samples

Assigned error_inj to 2'b11 if err_trig[N:1] == 1.

Yields total of 13 corrupts with all 256 inputs and outputs matching.

c) Consecutive bad bit[0]'s

A string of consecutive bad bit[0]'s depends on the interval of how often samples are taken. For an interval of every 8 samples, 4 corrupted bits are required for errors to start occurring, while for every 16 samples, 5 corrupted bits are needed. Beyond that, it can be observed that for every 32 samples, a sequence of 9 consecutive corrupted bits leads to errors in the Viterbi Decoder. Thus, it can be assumed that a general formula can be derived for the ratio of minimum number of consecutive bad bit[0]'s for a given sample which causes errors is $(2^{N-1} + 1) : 2^{N+1}$, where N is greater than 1. This can be checked within the Viterbi transmitter/receiver by changing the condition for errors to be injected to `word_ct[3:0] >= 0 && word_ct[3:0] <= 4` (using the example of 5 bits : sample of 16).

d) Consecutive bad bit[1]'s

It can be observed that the number of consecutive bad bit[1]'s needed for errors to occur in the Viterbi Decoder is the same as that of the number of consecutive bad bit[0]'s. So, the same general rule can be applied.

e) Consecutive sequence of both bad bit[1]'s and bit[0]'s

For a consecutive sequence of both bad bit[1]'s and bit[0]'s (2'b11), it can be observed that the minimum number of consecutive errors is approximately halved, rounding up, before bad bits are created. So, for an interval of every 8 samples, 2 corrupted bits are required, and for an interval of every 16 samples, 3 corrupted bits lead to bad bits being created, etc. So, in this case, the general formula for the ratio of minimum number of consecutive bad bits for a given sample to cause errors is $(2^{N-2} + 1) : 2^{N+1}$, where N is greater than 1. For example, for 3 bits : sample of 16, the changed condition would be `word_ct[3:0] >= 0 && word_ct[3:0] <= 2`.